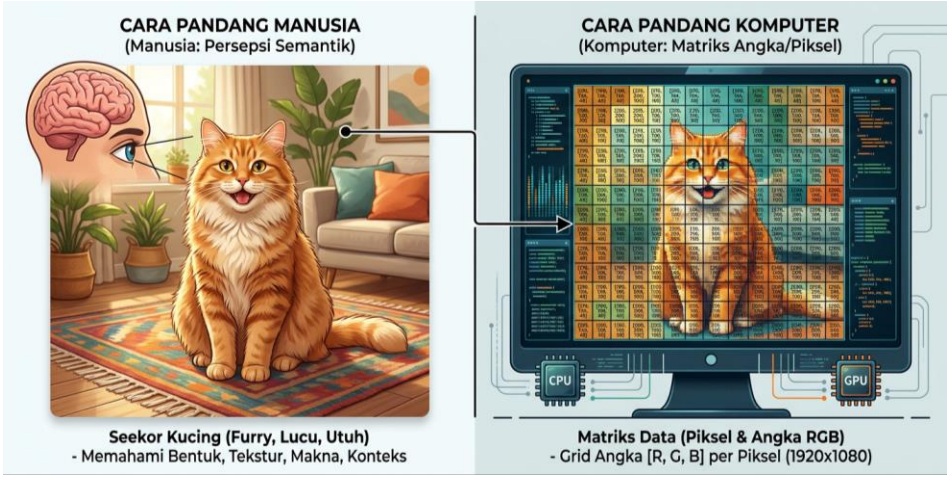
# MODUL 1 SISTEM MULTIMEDIA

# IMAGE FEATURE EXTRACTIONS

# PENDAHULUAN

Bagi sistem visual manusia, mengenali sebuah objek di dalam gambar adalah hal yang sangat intuitif dan terjadi dalam hitungan milidetik. Manusia dengan mudah dapat membedakan mana kucing dan mana anjing hanya dengan melihat sekilas. Namun, bagi sebuah komputer, gambar hanyalah representasi matematis berupa matriks dua dimensi atau tiga dimensi yang berisi nilai-nilai piksel intensitas cahaya dan warna. Memproses ribuan hingga jutaan piksel secara mentah (*raw data*) untuk mengenali suatu objek tidak hanya tidak efisien, tetapi juga menuntut beban komputasi yang sangat besar. Oleh karena itu, diperlukan suatu metode untuk menyaring informasi penting dari lautan piksel tersebut, sebuah proses fundamental yang dikenal sebagai **ekstraksi fitur (*feature extraction*)**.

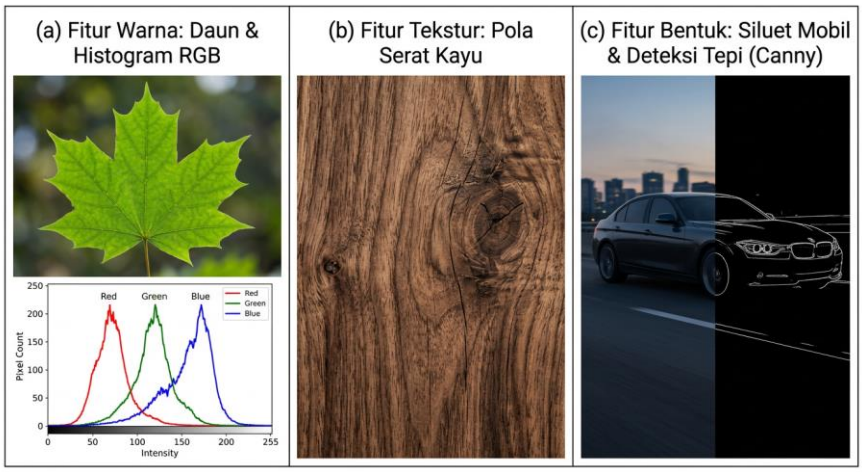



*Gambar 1: Ilustrasi perbedaan cara pandang manusia (melihat bentuk utuh objek) dibandingkan dengan komputer (melihat matriks angka/piksel).*

Secara definisi, ekstraksi fitur adalah teknik reduksi dimensi yang mengubah data mentah citra menjadi representasi matematis baru yang lebih padat (*compact*) tanpa menghilangkan informasi esensial yang terkandung di dalamnya. Fitur (*feature*) itu sendiri merupakan karakteristik unik, pola, atau atribut spesifik yang dapat membedakan satu objek dengan objek lainnya di dalam citra. Representasi fitur ini bisa berupa titik sudut (*corner*), tepi (*edge*), bercak (*blob*), pola tekstur, maupun distribusi statistik warna. Dengan mengekstrak fitur, kita mengubah gambar dari format piksel menjadi sekumpulan vektor (*feature vector*) yang jauh lebih ringan untuk diproses oleh algoritma selanjutnya.

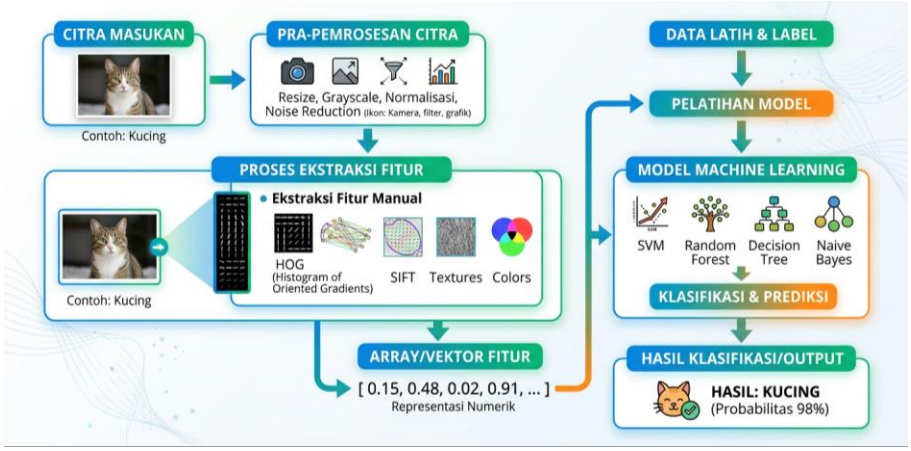
*Gambar 2: Contoh visualisasi hasil deteksi fitur pada gambar.*

Dalam pengolahan citra digital dan visi komputer (*computer vision*), ekstraksi fitur umumnya diklasifikasikan menjadi tiga kategori utama berdasarkan jenis informasi visual yang diekstrak. **Pertama, fitur warna (*color features*)**, yang berfokus pada distribusi warna menggunakan histogram atau momen warna. **Kedua, fitur tekstur (*texture features*)**, yang menganalisis kekasaran, kehalusan, atau pola berulang pada permukaan objek, seringkali menggunakan metode seperti *Gray Level Co-occurrence Matrix* (GLCM). **Ketiga, fitur bentuk dan tepi (*shape and edge features*)**, yang mendeteksi batas-batas geometris objek menggunakan metode seperti *Canny Edge Detection*, *Histogram of Oriented Gradients* (HOG), atau algoritma pendeteksi lainnya.




*Gambar 3: Kolase 3 gambar yang mewakili jenis fitur: (a) Gambar daun beserta grafik histogram warnanya, (b) Gambar pola serat kayu untuk tekstur, dan (c) Gambar siluet mobil dengan visualisasi deteksi tepi (Canny).*

Dalam ekosistem kecerdasan buatan (*Artificial Intelligence*), ekstraksi fitur memainkan peran yang sangat krusial. Algoritma *Machine Learning* tradisional, seperti *Support Vector Machine* (SVM) atau *K-Nearest Neighbors* (KNN), tidak dapat 'melihat' gambar secara langsung; mereka sangat bergantung pada kualitas vektor fitur yang diekstrak secara manual (*hand-crafted features*). Terdapat pepatah terkenal dalam ilmu data: *"Garbage in, garbage out"*. Sebaik apapun algoritma klasifikasi yang digunakan, jika fitur yang diekstrak dari gambar tidak representatif dan tidak mampu membedakan antar kelas objek, maka akurasi sistem akan tetap rendah. Oleh karena itu, kemampuan merancang dan memilih metode ekstraksi fitur yang tepat adalah kompetensi wajib bagi seorang *computer vision engineer*.




*Gambar 4: Diagram blok/Flowchart dari pipeline Machine Learning tradisional.*

Melalui modul praktikum ini, mahasiswa akan diajak untuk beralih dari sekadar pemahaman teoretis menuju implementasi praktis. Mahasiswa akan mempelajari cara kerja berbagai algoritma ekstraksi fitur secara matematis dan menerapkannya menggunakan bahasa pemrograman. Dengan mengekstrak berbagai jenis fitur pada citra uji, mahasiswa diharapkan mampu menganalisis pengaruh setiap metode terhadap representasi data, serta memahami kapan dan mengapa suatu metode ekstraksi fitur lebih unggul dibandingkan metode lainnya dalam menyelesaikan studi kasus deteksi atau pengenalan objek tertentu.

## DEPENDENSI

### CODE

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
from scipy.stats import skew, kurtosis # Color Statistics
from skimage.feature import local_binary_pattern # LBP
from skimage.feature import graycomatrix, graycoprops # GLCM
from skimage.feature import hog # HOG

# Transfer Learning VGG16
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import Model
```

## LOAD DATA

### CODE

```
nama_file = "/content/Jepang_Disney.png" # path file

gambar_bgr = cv2.imread(nama_file)

# OpenCV membaca gambar dengan format BGR (Biru, Hijau, Merah).
# Kita perlu mengubahnya ke RGB (Merah, Hijau, Biru) agar warnanya benar
saat ditampilkan.
gambar_asli = cv2.cvtColor(gambar_bgr, cv2.COLOR_BGR2RGB)

# Kita juga membuat versi Hitam Putih (Grayscale)
gambar_abu = cv2.cvtColor(gambar_bgr, cv2.COLOR_BGR2GRAY)
```

## COLOR HISTOGRAM

Color Histogram merupakan salah satu representasi statistik yang paling fundamental dalam pengolahan citra digital. Secara sederhana, color histogram adalah penghitungan distribusi frekuensi kemunculan warna-warna yang terdapat dalam sebuah citra. Berbeda dengan histogram pada citra grayscale yang hanya memiliki satu dimensi, color histogram biasanya bersifat multidimensi karena melibatkan beberapa saluran warna sekaligus.

Pada citra grayscale, histogram didefinisikan secara matematis sebagai berikut:

$$h(i) = \sum_{\{x=1\}}^{\{M\}} \sum_{\{y=1\}}^{\{N\}} \delta(I(x,y) - i), \qquad i = 0, 1, 2, \dots, L-1$$

di mana $M \times N$ adalah ukuran citra, $L$ biasanya bernilai 256 untuk citra 8-bit, dan $\delta(\cdot)$ adalah fungsi Kronecker delta. Fungsi ini menghitung berapa kali intensitas nilai $i$ muncul dalam citra.

*Gambar 2.1: Ilustrasi citra asli beserta histogram grayscale dan color histogram-nya*

Untuk citra berwarna, khususnya dalam ruang warna RGB, color histogram menjadi sebuah histogram tiga dimensi. Setiap piksel memiliki nilai warna $(R, G, B)$. Histogram RGB tiga dimensi dapat dinyatakan dengan persamaan:

$$H(r,g,b) = \sum_{x=1}^{M} \sum_{y=1}^{N} \delta(I_R(x,y) - r) \cdot \delta(I_G(x,y) - g) \cdot \delta(I_B(x,y) - b)$$

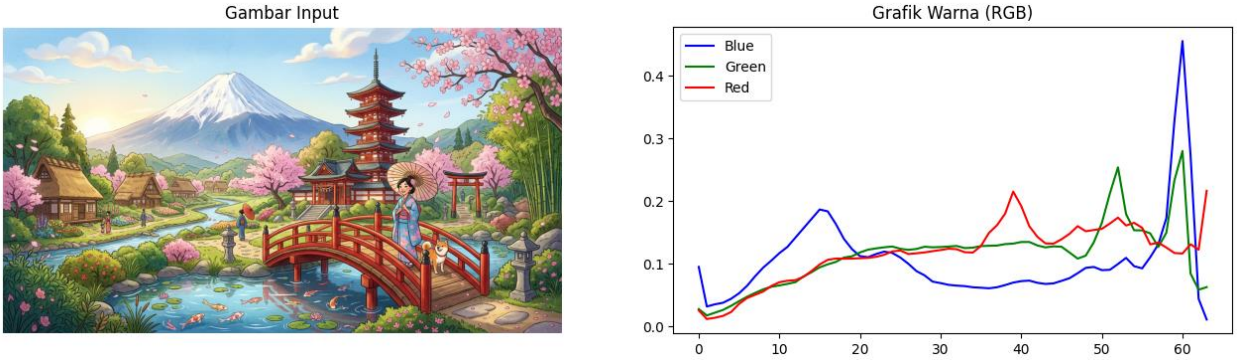
dengan $r$, $g$, dan $b$ masing-masing merepresentasikan tingkat kecerahan pada saluran merah, hijau, dan biru. Tanpa kuantisasi, jika setiap saluran memiliki 256 tingkat, maka total bin yang dihasilkan mencapai $256^3 = 16.777.216$ bin, yang secara komputasi sangat tidak efisien.

Oleh karena itu, proses kuantisasi (quantization) hampir selalu dilakukan. Misalnya, dengan mengurangi setiap saluran menjadi hanya 8 tingkat (8×8×8), jumlah bin menjadi 512. Proses ini dilakukan dengan membagi nilai asli setiap saluran dengan faktor tertentu, seperti $r' = \lfloor R/32 \rfloor$ untuk mendapatkan 8 bin per saluran. Setelah dikuantisasi, histogram menjadi lebih ringkas namun tetap mampu merepresentasikan distribusi warna secara keseluruhan.

Agar histogram dapat dibandingkan antar citra dengan ukuran yang berbeda, biasanya dilakukan normalisasi. Histogram ternormalisasi dihitung dengan membagi setiap nilai frekuensi dengan total jumlah piksel, sehingga:

$$p(r, g, b) = \frac{H(r, g, b)}{M \times N}$$

Hasilnya merupakan distribusi probabilitas warna di mana jumlah keseluruhan probabilitas sama dengan 1.




*Gambar 2.2: Histogram per channel (R, G, B)*

Selain histogram tiga dimensi, sering juga digunakan histogram per saluran (marginal histograms). Histogram saluran merah, misalnya, dihitung dengan cara:

$$H_R(r) = \sum_g \sum_b H(r, g, b)$$

Pendekatan ini menghasilkan tiga histogram satu dimensi terpisah untuk saluran R, G, dan B. Meskipun kehilangan informasi hubungan antar saluran, pendekatan ini jauh lebih efisien dan sering memberikan hasil yang cukup baik untuk banyak aplikasi.

Pemilihan ruang warna sangat berpengaruh terhadap kualitas color histogram. Ruang RGB bersifat sederhana tetapi tidak sesuai dengan persepsi manusia. Oleh karena itu, banyak peneliti lebih memilih ruang warna HSV atau Lab. Ruang Lab khususnya dianggap unggul karena bersifat perceptually uniform, artinya jarak Euclidean antara dua warna dalam ruang ini lebih sesuai dengan perbedaan yang dirasakan oleh mata manusia.

Meskipun sangat berguna, color histogram memiliki keterbatasan utama yaitu tidak menyimpan informasi spasial. Histogram hanya melihat distribusi warna secara keseluruhan tanpa mempedulikan posisi warna tersebut berada di mana dalam citra. Untuk mengatasi kelemahan ini, teknik seperti Spatial Pyramid Matching atau Local Color Histogram sering dikombinasikan dengan color histogram standar.

**CODE**

```
plt.figure(figsize=(16, 4))

# --- Tampilan 1: Gambar Asli ---
plt.subplot(1, 2, 1)
plt.imshow(gambar_asli)
plt.title("Gambar Input")
plt.axis("off")

# --- Tampilan 2: Grafik Warna (Histogram) ---
plt.subplot(1, 2, 2)
plt.title("Grafik Warna (RGB)")

# Kita buat daftar urutan warna OpenCV yaitu B, G, R (Blue, Green, Red)
kode_warna = ['b', 'g', 'r']
nama_warna = ['Blue', 'Green', 'Red']

jumlah_bin = 64 # Seberapa detail grafiknya (makin besar makin rapat)
kumpulan_hasil = []

# Kita hitung satu per satu: 0 untuk Blue, 1 untuk Green, 2 untuk Red
for i in range(3):

    # Menghitung jumlah warna (Blue/Green/Red) pada gambar
    grafik = cv2.calcHist([gambar_bgr], [i], None, [jumlah_bin], [0, 256])

    # Normalisasi agar skalanya hanya 0 sampai 1
    grafik_rapi = cv2.normalize(grafik, grafik).flatten()

    # Extend
    kumpulan_hasil.extend(grafik_rapi)

    # Gambar garis grafiknya (sesuai warna)
    plt.plot(grafik_rapi, color=kode_warna[i], label=nama_warna[i])

plt.legend()
plt.show()
```

## COLOR STATISTICS (MEAN, STD, SKEWNESS, KURTOSIS)

Color Statistics adalah salah satu metode ekstraksi fitur yang paling fundamental dalam pemrosesan citra dan computer vision. Metode ini menganalisis distribusi nilai intensitas piksel pada setiap channel warna (biasanya RGB, atau kadang HSV/Lab) menggunakan empat ukuran statistik utama: Mean, Standard Deviation, Skewness, dan Kurtosis. Keempat statistik ini bersama-sama memberikan deskripsi yang sangat lengkap tentang karakteristik warna suatu gambar, mulai dari tingkat kecerahan rata-rata hingga bentuk distribusi nilai warna.

Mean (Rata-rata) merupakan ukuran tendensi sentral yang paling sederhana namun sangat penting. Pada sebuah channel warna (misalnya channel Red), mean dihitung sebagai nilai rata-rata intensitas semua piksel:

$$\mu = \frac{1}{N}\sum_{i=1}^{N} x_i$$

di mana $(x_i)$ adalah nilai intensitas piksel ke-$(i)$ (berkisar 0–255 untuk citra 8-bit) dan $(N)$ adalah total jumlah piksel. Dalam konteks warna, mean mencerminkan tingkat kecerahan atau dominasi channel tersebut. Mean Red yang tinggi menandakan gambar cenderung berwarna merah/merah muda, sedangkan mean yang rendah di semua channel menunjukkan gambar gelap (low brightness).

Standard Deviation (Std) mengukur seberapa tersebar atau bervariasinya nilai piksel dari nilai mean-nya. Rumus matematisnya adalah:

$$\sigma = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2}$$

(atau menggunakan pembagi $N - 1$ untuk sampel). Nilai Std yang rendah menunjukkan warna relatif homogen atau datar (misalnya langit biru cerah yang seragam), sedangkan Std yang tinggi menandakan variasi warna yang besar, tekstur yang kompleks, atau transisi warna yang tajam. Std sering digunakan sebagai indikator kontras warna pada suatu channel.

Skewness (Kemiringan Distribusi) mengukur tingkat asimetri distribusi nilai piksel terhadap mean. Rumusnya adalah:

$$\text{Skewness} = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^3}{\sigma^3}$$

Interpretasi skewness sangat penting:

- Skewness > 0 (positive skew): ekor distribusi berada di sebelah kanan → banyak piksel gelap dan sedikit piksel sangat terang (contoh: foto dengan background gelap dan highlight terang).
- Skewness < 0 (negative skew): ekor di sebelah kiri → banyak piksel terang dan sedikit piksel sangat gelap.
- Skewness ≈ 0: distribusi relatif simetris.

Dalam analisis warna, skewness sangat berguna untuk mendeteksi apakah suatu warna didominasi oleh nilai rendah atau tinggi.

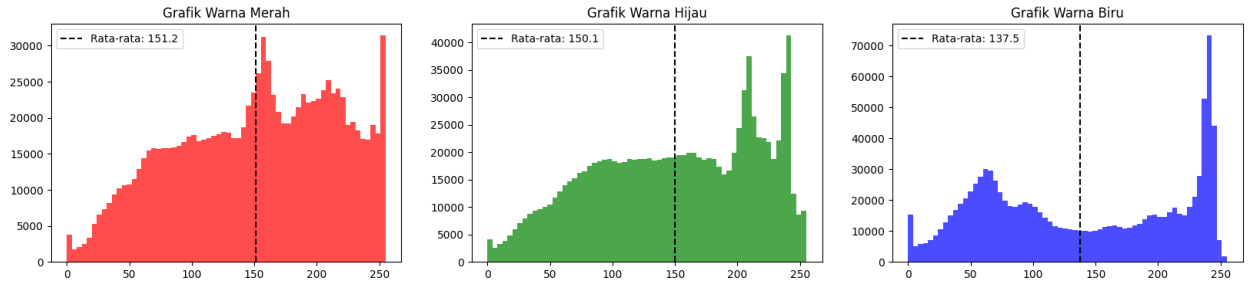
Kurtosis menggambarkan "keruncingan" dan ketebalan ekor (tailedness) distribusi dibandingkan dengan distribusi normal. Rumus kurtosis (excess kurtosis) yang paling umum digunakan adalah:

$$\text{Kurtosis} = \frac{\frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^4}{\sigma^4} - 3$$

Interpretasi:

- Kurtosis > 0 (Leptokurtic): distribusi lebih runcing dengan ekor yang lebih berat → terdapat banyak piksel ekstrem (sangat gelap atau sangat terang).
- Kurtosis < 0 (Platykurtic): distribusi lebih datar dengan ekor yang lebih ringan.
- Kurtosis ≈ 0 (Mesokurtic): mendekati distribusi normal.

Dalam color statistics, kurtosis yang tinggi pada suatu channel biasanya menandakan adanya area-area dengan intensitas ekstrem, seperti pantulan cahaya yang kuat, bayangan pekat, atau saturasi warna yang sangat tinggi.




*Gambar 3.1: Mean Color Statistics*

Keempat statistik ini biasanya dihitung secara terpisah untuk setiap channel warna, sehingga untuk citra RGB akan dihasilkan 12 fitur (4 statistik × 3 channel). Kombinasi keempatnya memberikan "sidik jari warna" yang sangat powerful untuk tugas-tugas seperti klasifikasi gambar, retrieval berbasis konten (CBIR), deteksi objek, dan analisis estetika visual.

**CODE**

```
# Gambar RGB itu terdiri dari 3 lapis: Merah (0), Hijau (1), Biru (2).
# Kita ambil lapisannya satu per satu, lalu ratakan angkanya pakai
.flatten()
lapisan_merah = gambar_asli[:, :, 0].flatten()
lapisan_hijau = gambar_asli[:, :, 1].flatten()
lapisan_biru  = gambar_asli[:, :, 2].flatten()

# Kita kumpulkan jadi satu paket agar mudah diulang (loop)
data_warna = [
    ("Merah", lapisan_merah, 'red'),
    ("Hijau", lapisan_hijau, 'green'),
    ("Biru",  lapisan_biru,  'blue')
]

keranjang_fitur = []            # Tempat menyimpan semua hasil akhir angka
plt.figure(figsize=(20, 4))     # Siapkan kanvas panjang untuk 3 grafik
posisi_grafik = 1               # Mulai dari kotak grafik pertama

print("Laporan Statistik Warna:")
print("--------------------------------------------------")

# Kita cek satu-satu: Merah dulu, lalu Hijau, lalu Biru
for nama, data_piksel, kode_warna in data_warna:

    # --- A. MENGHITUNG STATISTIK ---
    rata_rata = np.mean(data_piksel)    # Kecerahan rata-rata
    simpangan = np.std(data_piksel)     # Jarak sebaran warna (kontras)
    kemiringan = skew(data_piksel)      # Apakah warnanya condong
gelap/terang
    keruncingan = kurtosis(data_piksel)# Apakah ada satu warna yang paling
mendominasi

    # Extend 4 angka ini ke keranjang kita
    keranjang_fitur.extend([rata_rata, simpangan, kemiringan, keruncingan])
```

```
    # --- B. MENAMPILKAN TEKS ---
    print(f"Warna {nama}:")
    print(f" - Rata-rata   : {rata_rata:.2f}")
    print(f" - Standar Dev : {simpangan:.2f}")
    print(f" - Skewness    : {kemiringan:.2f}")
    print(f" - Kurtosis    : {keruncingan:.2f}")
    print("--------------------------------------------------")

    # --- C. MENGGAMBAR GRAFIK ---
    plt.subplot(1, 3, posisi_grafik)
    plt.hist(data_piksel, bins=64, color=kode_warna, alpha=0.7)
    plt.title(f"Grafik Warna {nama}")

    # Tarik garis lurus putus-putus untuk menandai nilai rata-rata
    plt.axvline(rata_rata, color='black', linestyle='--', label=f'Rata-rata:
{rata_rata:.1f}')
    plt.legend()

    posisi_grafik += 1 # Maju ke kotak grafik selanjutnya

# Tampilkan ke layar
plt.show()

hasil_akhir = np.array(keranjang_fitur)
print(f"Selesai! Total ada {len(hasil_akhir)} angka yang didapat.")
print("(4 statistik dikali 3 warna = 12 angka)")
```

## LOCAL BINARY PATTERN (LBP) - TEKSTUR

Local Binary Pattern (LBP) adalah salah satu metode ekstraksi fitur tekstur yang sangat populer dalam bidang pengolahan citra digital dan visi komputer. Pada intinya, LBP dirancang untuk mengenali dan mengklasifikasikan pola tekstur lokal pada sebuah gambar dengan cara yang sangat efisien secara komputasi. Konsep dasar LBP adalah membandingkan nilai intensitas piksel pusat dengan nilai intensitas piksel-piksel yang ada di sekitarnya (tetangganya). Hasil perbandingan ini kemudian diubah menjadi bilangan biner yang pada akhirnya merepresentasikan karakteristik struktur mikro dari area lokal tersebut, seperti tepian (edge), sudut (corner), bintik (spot), atau area datar (flat).

Dalam bentuk aslinya, operator LBP bekerja pada area neighborhood atau ketetanggaan berukuran $3 \times 3$ piksel. Misalkan kita memiliki sebuah piksel pusat dengan intensitas $g_c$, dan dikelilingi oleh 8 piksel tetangga dengan intensitas $g_p$ di mana $p = 0, 1, \ldots, 7$. LBP menerapkan proses ambang batas (thresholding) pada setiap piksel tetangga menggunakan nilai piksel pusat sebagai nilai ambangnya. Jika intensitas piksel tetangga lebih besar atau sama dengan piksel pusat, maka diberi nilai biner 1; sebaliknya, jika lebih kecil, diberi nilai biner 0. Rangkaian 8 nilai biner ini kemudian dibaca secara melingkar (searah atau berlawanan arah jarum jam) untuk membentuk sebuah angka desimal.

Secara matematis, operator LBP dasar dapat dirumuskan dengan persamaan berikut:

$$LBP = \sum_{p=0}^{7} s(g_p - g_c) 2^p$$

Dalam persamaan tersebut, $s(x)$ adalah fungsi langkah Heaviside (Heaviside step function) yang didefinisikan sebagai:

$$s(x) = \begin{cases} 1, & \text{jika x} \geq \text{y} \\ 0, & \text{jika x} < \text{y} \end{cases}$$

Perkalian dengan $2^p$ berfungsi untuk memberikan bobot biner pada setiap posisi piksel tetangga, sehingga mengubah deretan angka biner menjadi sebuah nilai desimal tunggal dalam rentang 0 hingga 255. Nilai desimal inilah yang menggantikan nilai piksel pusat pada citra hasil transformasi LBP.

Seiring perkembangannya, algoritma LBP asli memiliki kelemahan karena hanya terbatas pada area $3 \times 3$, sehingga tidak dapat menangkap tekstur pada skala yang berbeda (makro-tekstur). Oleh karena itu, diperkenalkan Extended LBP atau Circular LBP (ditulis sebagai $LBP_{P,R}$) yang menggunakan lingkungan melingkar. LBP jenis ini memungkinkan penggunaan jumlah piksel tetangga sebanyak $P$ pada jarak radius sebesar $R$ dari piksel pusat. Koordinat $(x_p, y_p)$ dari setiap piksel tetangga $P$ terhadap koordinat piksel pusat $(x_c, y_c)$ ditentukan menggunakan rumus trigonometri:

$$x_p = x_c + R\cos\left(\frac{2\pi p}{P}\right)$$

$$y_p = y_c - R\sin\left(\frac{2\pi p}{P}\right)$$

Karena nilai koordinat $(x_p, y_p)$ yang dihasilkan dari persamaan di atas tidak selalu berupa bilangan bulat (piksel tidak selalu jatuh tepat di tengah kotak grid piksel gambar), algoritma LBP menggunakan teknik interpolasi bilinear untuk mengestimasi nilai intensitas piksel tetangga yang posisinya berada di antara koordinat piksel aktual.

Selain variasi skala, tantangan lain dari LBP melingkar adalah ledakan dimensi. Jika kita menggunakan $P = 16$, maka akan ada $2^{16} = 65536$ kemungkinan pola, yang membuat pembuatan histogram menjadi sangat besar dan rentan terhadap noise. Untuk mengatasi ini, diperkenalkan konsep Uniform LBP ($LBP_{P,R}^{u2}$). Sebuah pola LBP disebut "seragam" (uniform) jika pola binernya memiliki paling banyak dua transisi bit (dari 0 ke 1, atau dari 1 ke 0) ketika dibaca secara melingkar. Derajat keseragaman $U$ dihitung dengan persamaan matematis:

$$U(LBP_{P,R}) = |s(g_{P-1} - g_c) - s(g_0 - g_c)| + \sum_{p=1}^{P-1} |s(g_p - g_c) - s(g_{p-1} - g_c)|$$

Sebuah pola dikategorikan seragam jika $U \leq 2$. Pola-pola seragam ini mewakili fitur-fitur penting seperti ujung garis, sudut, atau bintik. Dalam klasifikasi, setiap pola seragam diberikan nilai desimal yang terpisah, sementara seluruh pola yang tidak seragam (non-uniform) dikelompokkan ke dalam satu nilai (satu bin yang sama). Hal ini secara drastis mengurangi jumlah fitur, dari $2^P$ menjadi $P(P-1) + 3$ pola saja.

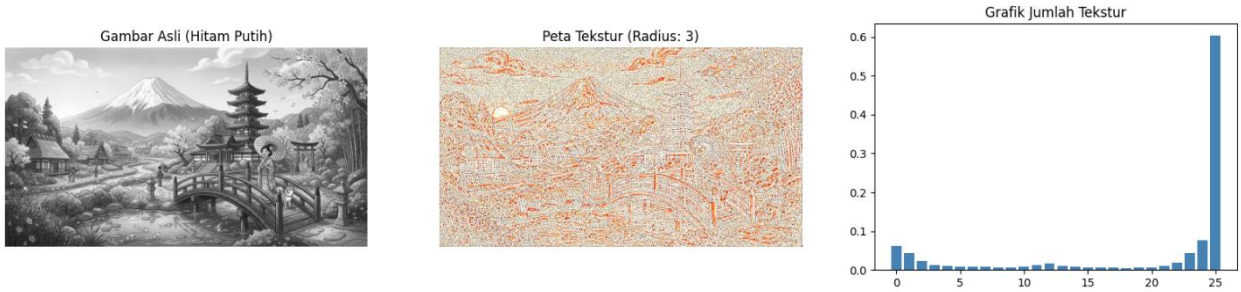
Langkah terakhir dalam ekstraksi fitur menggunakan LBP adalah pembentukan Histogram. Setelah citra asli ditransformasi menjadi "citra LBP" (di mana setiap piksel kini bernilai desimal LBP), fitur yang sebenarnya digunakan untuk machine learning adalah distribusi dari nilai-nilai LBP tersebut di seluruh gambar atau di dalam blok-blok gambar tertentu. Histogram $H$ direpresentasikan oleh:

$$H(k) = \sum_{i=1}^{N} \sum_{j=1}^{M} f(LBP(i,j), k)\,, \quad \text{untuk } k \in [0, K]$$

Di mana gambar berukuran $N \\times M$, $K$ adalah nilai maksimal dari LBP (misalnya 255 untuk LBP dasar, atau 58 untuk $LBP_{8,R}^{u2}$), dan fungsi $f(x, y)$ didefinisikan sebagai:

$$f(x, y) = \begin{cases} 1, & \text{jika x = y} \\ 0, & \text{jika x} \neq \text{y} \end{cases}$$

Histogram ini merangkum frekuensi kemunculan setiap struktur mikro pada gambar. Gabungan dari histogram-histogram ini membentuk vektor fitur (feature vector) yang kemudian diumpankan ke dalam algoritma pengklasifikasi (seperti SVM atau KNN) untuk tugas-tugas seperti pengenalan wajah (face recognition), deteksi ekspresi wajah, atau analisis citra medis. LBP sangat disukai karena perhitungannya yang sangat cepat dan sifat matematis ambang batasnya $s(g_p - g_c)$ yang membuatnya sangat invarian (kebal) terhadap perubahan pencahayaan secara monotonik yang merata di seluruh area citra.



**CODE**

```
jari_jari = 3                       # Seberapa lebar area piksel yang mau
dicek
jumlah_titik = 8  jari_jari      # Jumlah piksel tetangga yang dilihat (8 x
3 = 24)

# Komputer mengecek pola tekstur dari gambar hitam putih kita
gambar_tekstur = local_binary_pattern(gambar_abu, jumlah_titik, jari_jari,
method='uniform')

# Kita cari tahu ada berapa macam pola tekstur yang ditemukan
jumlah_macam_pola = int(gambar_tekstur.max() + 1)

# Kita hitung jumlah masing-masing pola ke dalam bentuk grafik/histogram
# density=True artinya kita ubah angkanya jadi persentase (0 sampai 1),
bukan jumlah mutlak
angka_grafik, batas_kotak = np.histogram(gambar_tekstur.flatten(),
bins=jumlah_macam_pola, density=True)


plt.figure(figsize=(20, 4)) # Siapkan kanvas memanjang

# --- Tampilan 1: Gambar Hitam Putih Asli ---
plt.subplot(1, 3, 1)
plt.imshow(gambar_abu, cmap='gray')
plt.title("Gambar Asli (Hitam Putih)")
plt.axis("off")

# --- Tampilan 2: Gambar Hasil Deteksi Tekstur ---
plt.subplot(1, 3, 2)
# cmap='hot' membuat warna teksturnya jadi seperti kamera pendeteksi panas
plt.imshow(gambar_tekstur, cmap='hot')
plt.title(f"Peta Tekstur (Radius: {jari_jari})")
plt.axis("off")

# --- Tampilan 3: Grafik Batang (Histogram LBP) ---
plt.subplot(1, 3, 3)
urutan_bawah = range(len(angka_grafik)) # Bikin nomor urut 0, 1, 2, dst
untuk posisi bawah
plt.bar(urutan_bawah, angka_grafik, color='steelblue')
plt.title("Grafik Jumlah Tekstur")

plt.show()

print("Informasi Tekstur (LBP):")
print(f" - Jangkauan (Radius)   : {jari_jari} piksel")
print(f" - Titik yang dicek     : {jumlah_titik} titik per piksel")
print(f" - Total jenis pola     : {len(angka_grafik)} angka")
```

## GRAY-LEVEL CO-OCCURRENCE MATRIX (GLCM)

Gray-Level Co-occurrence Matrix (GLCM) adalah salah satu metode statistik paling klasik dan kuat untuk analisis tekstur dalam pengolahan citra digital, yang pertama kali diperkenalkan oleh Robert Haralick pada tahun 1973. Berbeda dengan histogram biasa yang merupakan metode statistik orde pertama (hanya menghitung frekuensi kemunculan piksel secara individu), GLCM adalah metode statistik orde kedua. Artinya, GLCM mempertimbangkan hubungan spasial (tata letak) antara dua piksel. Konsep utamanya adalah menghitung seberapa sering sepasang piksel dengan nilai intensitas kecerahan ( gray-level) tertentu muncul secara berdampingan di dalam sebuah citra, berdasarkan jarak dan sudut yang telah ditentukan. Dari matriks yang terbentuk ini, kita dapat mengekstrak berbagai fitur numerik yang menggambarkan kekasaran, kehalusan, atau pola repetitif dari sebuah tekstur gambar.

Untuk membentuk sebuah matriks GLCM, kita memerlukan sebuah citra keabuan ( grayscale) $I$ berukuran $N_x \times N_y$ dengan jumlah tingkat keabuan sebesar $G$ (misalnya, jika gambar 8-bit, maka $G=256$). Pembentukan matriks ini sangat bergantung pada dua parameter spasial: jarak antar piksel $d$ dan sudut orientasi $\theta$. Sudut yang umum digunakan adalah $0^\circ, 45^\circ, 90^\circ,$ dan $135^\circ$. Matriks GLCM, yang dilambangkan dengan $P_{d, \theta}$, adalah sebuah matriks persegi berukuran $G \times G$. Nilai pada setiap sel matriks $P(i, j)$ merepresentasikan jumlah kemunculan (frekuensi) di mana piksel dengan intensitas $i$ bertetangga dengan piksel berintensitas $j$ pada jarak $d$ dan sudut $\theta$. Secara matematis, hal ini didefinisikan sebagai:

$$
\begin{aligned}
P_{d,\theta}(i,j) &= \sum_{x=1}^{N_x} \sum_{y=1}^{N_y} \begin{cases} 1, & \text{jika } I(x,y) \\ & \text{dan } I(x+\Delta x, y+\Delta y) \\ & \text{untuk kondisi lainnya} \end{cases} \\
&= i \\
&= j \\
&= 0, \quad \text{untuk kondisi lainnya}
\end{aligned}
$$

Di mana $\Delta x$ dan $\Delta y$ adalah pergeseran ( offset) koordinat yang nilainya bergantung pada jarak $d$ dan sudut $\theta$. Untuk mempercepat komputasi dan mengurangi ukuran matriks yang sangat besar ($256 \times 256$), citra sering kali di-kuantisasi terlebih dahulu menjadi tingkat keabuan yang lebih kecil, misalnya $G=8$ atau $G=16$.

Setelah matriks frekuensi $P_{d, \theta}$ terbentuk, langkah esensial berikutnya adalah melakukan normalisasi matriks. Matriks GLCM mentah berisi jumlah hitungan yang nilainya sangat bergantung pada ukuran citra. Agar fitur yang diekstrak nantinya kebal terhadap perbedaan dimensi citra (scale invariant), matriks tersebut diubah menjadi matriks probabilitas $p(i, j)$. Normalisasi dilakukan dengan membagi setiap elemen matriks dengan total jumlahan seluruh elemen di dalam matriks tersebut. Persamaan normalisasinya adalah:

$$p(i,j) = \frac{P(i,j)}{\sum_{i=0}^{G-1}\sum_{j=0}^{G-1} P(i,j)}$$

Hasil dari normalisasi ini menjamin bahwa total jumlahan seluruh probabilitas di dalam matriks $p(i, j)$ adalah sama dengan $1$. Matriks probabilitas inilah yang kemudian digunakan untuk menghitung fitur-fitur tekstur Haralick.

Fitur pertama yang paling sering diekstrak dari GLCM adalah Contrast (Kontras) dan Energy (Energi). Kontras mengukur variasi intensitas lokal atau perbedaan derajat keabuan antar piksel di dalam citra. Fitur ini akan bernilai tinggi jika gambar memiliki tekstur yang kasar atau banyak tepi yang tajam. Persamaan Kontras memberikan bobot kuadrat $(i - j)^2$ pada probabilitas, sehingga piksel dengan intensitas yang sangat berbeda (berada jauh dari diagonal utama matriks GLCM) akan memberikan kontribusi yang besar:

$$\text{Contrast} = \sum_{i=0}^{G-1}\sum_{j=0}^{G-1}(i-j)^2 p(i,j)$$

Sementara itu, Energi (sering juga disebut Angular Second Moment atau ASM) mengukur tingkat keseragaman ( uniformity) dari tekstur. Energi akan bernilai maksimum (yaitu 1) jika gambar memiliki warna yang sepenuhnya konstan, dan bernilai tinggi jika pola tekstur sangat teratur atau periodik. Formulanya menjumlahkan kuadrat dari setiap nilai probabilitas:

$$\text{Energy} = \sum_{i=0}^{G-1}\sum_{j=0}^{G-1} p(i,j)^2$$

Fitur penting lainnya adalah Homogeneity (Homogenitas) dan Entropy (Entropi). Homogenitas mengukur seberapa dekat distribusi elemen-elemen di dalam GLCM terhadap diagonal utama matriks. Ini berkebalikan dengan kontras; homogenitas akan bernilai tinggi jika gambar tidak banyak berubah (piksel tetangganya memiliki nilai intensitas yang sama atau sangat mirip). Persamaannya adalah:

$$\text{Homogeneity} = \sum_{i=0}^{G-1}\sum_{j=0}^{G-1}\frac{p(i,j)}{1+(i-j)^2}$$

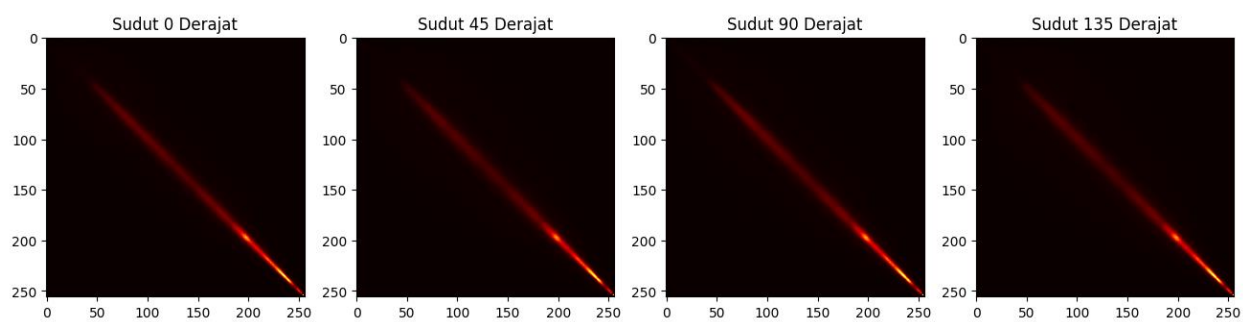
Di sisi lain, Entropi adalah ukuran statistik dari tingkat keacakan ( randomness) atau kompleksitas dari tekstur gambar. Tekstur yang sangat kompleks, acak, dan tidak memiliki struktur yang jelas (seperti noise atau derau) akan menghasilkan nilai entropi yang besar. Formula entropi mengambil konsep dasar dari teori informasi Shannon:

$$\text{Entropy} = -\sum_{i=0}^{G-1}\sum_{j=0}^{G-1} p(i,j)\log_2 p\,(i,j)$$

Fitur terakhir yang krusial adalah Correlation (Korelasi), yang mengukur seberapa kuat ketergantungan linier nilai keabuan antara sebuah piksel dengan tetangganya. Untuk menghitung fitur ini, kita memerlukan nilai rata-rata ( mean) disimbolkan dengan $\mu_x$ dan $\mu_y$, serta standar deviasi yang disimbolkan dengan $\sigma_x$ dan $\sigma_y$, yang dihitung dari probabilitas marginal baris dan kolom matriks GLCM. Nilai korelasi berkisar antara -1 hingga 1. Persamaan matematis untuk Korelasi adalah:

$$\text{Correlation} = \sum_{i=0}^{G-1}\sum_{j=0}^{G-1} \frac{(i-\mu_x)(j-\mu_y)p(i,j)}{\sigma_x\sigma_y}$$

Dalam praktiknya untuk aplikasi machine learning, untuk menghindari ketergantungan fitur pada arah rotasi gambar, ekstraksi fitur Haralick ini dihitung pada keempat sudut utama $(0°, 45°, 90°, 135°)$, lalu nilai rata-rata dari masing-masing fitur dihitung sebagai vektor fitur final (rotation invariant). Matriks GLCM dan turunan fiturnya ini sangat handal dan terus diandalkan dalam berbagai aplikasi berat seperti klasifikasi tutupan lahan pada citra satelit, deteksi cacat material di industri, hingga analisis pola tumor pada pencitraan medis (MRI atau CT-Scan).



**CODE**

```
jarak_langkah = [1, 3] # Kita lihat tetangga 1 langkah di sebelahnya, dan 3
langkah

# Sudut pandang (dalam matematika, sudut dihitung pakai pi/radian)
# 0 = Mendatar, pi/4 = Miring 45 derajat, pi/2 = Tegak 90 derajat, dst.
sudut_pandang = [0, np.pi/4, np.pi/2, 3np.pi/4]
nama_sudut = ['0 Derajat', '45 Derajat', '90 Derajat', '135 Derajat']

# Mesin akan mendata seluruh hubungan piksel berdasarkan jarak & sudut di
atas
matriks_glcm = graycomatrix(gambar_abu, distances=jarak_langkah,
angles=sudut_pandang,
                            levels=256, symmetric=True, normed=True)

# Menghitung 6 sifat tekstur
daftar_sifat = ['contrast', 'dissimilarity', 'homogeneity', 'energy',
'correlation', 'ASM']
keranjang_hasil = [] # Untuk menyimpan semua angka akhir

print("Laporan Sifat Tekstur (GLCM):")
print("--------------------------------------------------")

for sifat in daftar_sifat:
    # Ambil nilai sifat tersebut dari matriks GLCM
    nilai = graycoprops(matriks_glcm, sifat)

    # Hitung rata-rata dan penyimpangannya
    rata_rata = nilai.mean()
    simpangan = nilai.std()

    # Masukkan 2 angka ini ke keranjang kita
    keranjang_hasil.extend([rata_rata, simpangan])

    # Tampilkan di layar
    print(f"Sifat {sifat}:")
    print(f" - Rata-rata   : {rata_rata:.4f}")
    print(f" - Standar Dev : {simpangan:.4f}")
    print("--------------------------------------------------")

hasil_akhir = np.array(keranjang_hasil)
print(f"Total angka yang didapat: {len(hasil_akhir)} (6 sifat x 2 angka)")

# Siapkan 4 kotak kanvas karena kita punya 4 sudut pandang
plt.figure(figsize=(16, 4))

for i in range(4):
    plt.subplot(1, 4, i+1) # Urutan kotak ke 1, 2, 3, 4

    # Ambil gambar matriks untuk jarak langkah pertama (index 0) dan sudut
ke-i
    gambar_matriks = matriks_glcm[:, :, 0, i]

    # np.log1p digunakan sebagai 'filter' agar gambar lebih kelihatan terang
    plt.imshow(np.log1p(gambar_matriks), cmap='hot')
    plt.title(f"Sudut {nama_sudut[i]}")
```

```
plt.show()
```

## HU MOMENTS - BENTUK

Hu Moments (Momen Hu), yang diperkenalkan oleh Ming-Kuei Hu pada tahun 1962, adalah sekumpulan fitur statistik berupa deskriptor bentuk (shape descriptors) yang sangat fundamental dalam bidang pengolahan citra dan visi komputer. Tujuan utama dari ekstraksi fitur Hu Moments adalah untuk mendeskripsikan secara matematis bentuk geometris dari suatu objek di dalam citra sedemikian rupa sehingga nilai fitur yang dihasilkan bersifat invarian (tidak berubah). Invarian yang dimaksud di sini mencakup tiga hal utama: kebal terhadap pergeseran posisi objek (translasi), kebal terhadap perubahan ukuran objek (skala), dan kebal terhadap perputaran objek (rotasi). Oleh karena sifatnya ini, Hu Moments sangat populer digunakan untuk pencocokan bentuk, pengenalan karakter optik (OCR), dan analisis siluet objek.

Untuk memahami bagaimana Hu Moments terbentuk, kita harus memulai dari konsep Spatial Moments (momen spasial atau momen dasar). Dalam pengolahan citra digital dua dimensi, sebuah gambar direpresentasikan sebagai fungsi diskrit intensitas piksel $f(x, y)$ pada koordinat $(x, y)$. Momen spasial orde ke-$(p+q)$ dari sebuah gambar berukuran $M \times N$ didefinisikan secara matematis sebagai:

$$m_{pq} = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} x^p y^q f(x, y)$$

Di mana $p$ dan $q$ adalah bilangan bulat non-negatif yang menentukan orde momen (misalnya $m_{00}, m_{10}, m_{01}$). Nilai $m_{00}$ merupakan momen orde nol yang secara fisik merepresentasikan total massa citra (atau luas area objek jika citranya adalah biner/siluet). Kelemahan dari momen dasar ini adalah nilainya sangat bergantung pada posisi (koordinat) objek di dalam gambar. Jika objek bergeser, nilainya akan berubah total.

Agar fitur menjadi kebal terhadap pergeseran (translation invariant), kita perlu menggunakan Central Moments (momen pusat). Konsep dasarnya adalah memindahkan titik acuan perhitungan dari pojok kiri atas gambar ke titik pusat massa (centroid) objek itu sendiri. Koordinat pusat massa $(\bar{x}, \bar{y})$ dapat dihitung menggunakan momen dasar orde pertama dan orde ke-nol:

$$\bar{x} = \frac{m_{10}}{m_{00}}, \quad \bar{y} = \frac{m_{01}}{m_{00}}$$

Setelah titik centroid ditemukan, Momen Pusat orde ke-$(p+q)$, yang disimbolkan dengan $\mu_{pq}$, dihitung dengan mengukur jarak setiap piksel terhadap titik pusat massa tersebut:

$$\mu_{pq} = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} (x - \bar{x})^p (y - \bar{y})^q f(x, y)$$

Dengan menggunakan $\mu_{pq}$, letak objek di koordinat mana pun tidak akan memengaruhi hasil perhitungannya.

Langkah selanjutnya adalah memastikan fitur ini kebal terhadap perubahan ukuran (scale invariant). Objek yang sama namun direkam dari jarak dekat maupun jauh harus menghasilkan nilai fitur yang identik. Untuk mencapai hal ini, Momen Pusat harus dinormalisasi dengan cara membaginya menggunakan fungsi dari luas area/massa total ($\mu_{00}$). Ini disebut sebagai Normalized Central Moments (momen pusat ternormalisasi), yang disimbolkan dengan $\eta_{pq}$, dan dirumuskan sebagai:

$$\eta_{pq} = \frac{\mu_{pq}}{\mu_{00}^{\gamma}}$$

Di mana faktor pangkat $\gamma$ disesuaikan berdasarkan orde momen yang dihitung, menggunakan persamaan:

$$\gamma = \frac{p + q}{2} + 1 \quad \text{untuk } (p + q) \geq 2$$

Sebagai contoh, jika kita menghitung momen orde dua ($p+q=2$), maka nilai $\eta_{pq}$ didapatkan dengan membagi $\mu_{pq}$ dengan $\mu_{00}^2$.

Meskipun nilai $\eta_{pq}$ sudah kebal terhadap translasi dan skala, nilainya masih akan berubah jika objek diputar. Melalui teori invarian aljabar, Ming-Kuei Hu merumuskan 7 kombinasi polinomial khusus yang diekstrak dari momen pusat ternormalisasi orde dua dan orde tiga. Ketujuh kombinasi inilah yang secara resmi disebut sebagai Hu Moments ($h_1$ hingga $h_7$), yang dijamin secara matematis kebal terhadap rotasi (rotation invariant). Persamaan matematika dari ketujuh momen tersebut adalah:

$$h_1 = \eta_{20} + \eta_{02}$$

$$h_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2$$

$$h_3 = (\eta_{30} - 3\eta_{12})^2 + (3\eta_{21} - \eta_{03})^2$$

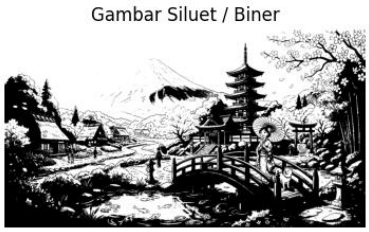
$$h_4 = (\eta_{30} + \eta_{12})^2 + (\eta_{21} + \eta_{03})^2$$

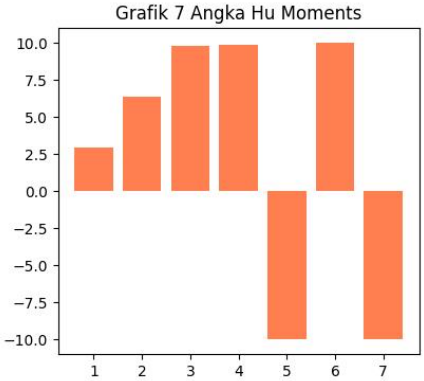
$$\begin{aligned} h_5 = {} & (\eta_{30} - 3\eta_{12})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] \\ & + (3\eta_{21} - \eta_{03})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] \end{aligned}$$

$$h_6 = (\eta_{20} - \eta_{02})[(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2] + 4\eta_{11}(\eta_{30} + \eta_{12})(\eta_{21} + \eta_{03})$$

$$h_7 = (3\eta_{21} - \eta_{03})(\eta_{30} + \eta_{12})[(\eta_{30} + \eta_{12})^2 - 3(\eta_{21} + \eta_{03})^2] - (\eta_{30} - 3\eta_{12})(\eta_{21} + \eta_{03})[3(\eta_{30} + \eta_{12})^2 - (\eta_{21} + \eta_{03})^2]$$

Momen $h_1$ hingga $h_6$ adalah invarian terhadap rotasi, skala, dan translasi. Sementara itu, momen ketujuh ($h_7$) memiliki keunikan sebagai skew invariant. Artinya, $h_7$ berguna untuk mendeteksi pantulan cermin (mirror image); nilainya akan sama persis namun berlawanan tanda (negatif/positif) jika sebuah objek dicerminkan. Dalam pengaplikasiannya pada algoritma machine learning, nilai Hu Moments yang dihasilkan bisa sangat kecil hingga mendekati nol, maupun sangat besar. Oleh karena itu, untuk menjaga stabilitas numerik, nilai dari ketujuh Hu Moments ini sering kali ditransformasikan menggunakan skala logaritmik sebelum digunakan sebagai vektor fitur, misalnya menggunakan rumus $H_i = -1 \times \text{sign}(h_i) \times \log_{10}(|h_i|)$. Transformasi ini menempatkan fitur-fitur tersebut dalam skala yang wajar untuk diproses oleh algoritma klasifikasi.



```python
# 1. MEMBUAT GAMBAR JADI HITAM PUTIH PEKAT (THRESHOLDING)
# Otsu Threshold bertugas menebak otomatis bagian mana objek (Putih) dan
mana background (Hitam)
nilai_batas, gambar_biner = cv2.threshold(gambar_abu, 0, 255,
cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 2. MENGHITUNG BENTUK (MOMENTS)
# OpenCV menghitung 'berat' dan titik tengah dari area warna putih
data_bentuk = cv2.moments(gambar_biner)

# Dari data di atas, kita saring menjadi 7 angka sakti (Hu Moments)
tujuh_angka_hu = cv2.HuMoments(data_bentuk).flatten()

# 3. MEMPERJELAS ANGKA (LOG TRANSFORM)
# Angka asli Hu Moments itu super duper kecil (bisa 0.0000000001).
# Kita harus pakai rumus Logaritma ini agar angkanya membesar dan mudah
dibaca komputer.
# (Jangan pusingkan rumusnya, ini cuma alat perbesar angka)
angka_diperjelas = -np.sign(tujuh_angka_hu)  np.log10(np.abs(tujuh_angka_hu)
+ 1e-10)

# 4. MENAMPILKAN HASIL AKHIR (TEKS)
print("Laporan 7 Angka Bentuk (Hu Moments):")
print("--------------------------------------------------")

for i in range(7):
    # Kita tampilkan 7 angkanya satu per satu
    print(f"Ciri Bentuk ke-{i+1}:")
    print(f" - Angka Asli   : {tujuh_angka_hu[i]:.8f}")
    print(f" - Angka Log    : {angka_diperjelas[i]:.4f}")
    print("--------------------------------------------------")

print("Selesai! Total ada 7 angka sakti untuk mengenali bentuk.")

# 5. MENGGAMBAR KE LAYAR
plt.figure(figsize=(15, 4)) # Siapkan 3 kanvas

# --- Tampilan 1: Gambar Abu-abu Asli ---
plt.subplot(1, 3, 1)
plt.imshow(gambar_abu, cmap='gray')
plt.title("Gambar Asli")
plt.axis("off")

# --- Tampilan 2: Gambar Siluet (Threshold Otsu) ---
plt.subplot(1, 3, 2)
plt.imshow(gambar_biner, cmap='gray')
plt.title("Gambar Siluet / Biner")
plt.axis("off")

# --- Tampilan 3: Grafik Batang ke-7 Angka Hu ---
plt.subplot(1, 3, 3)
urutan_bawah = range(1, 8) # Nomor 1 sampai 7
plt.bar(urutan_bawah, angka_diperjelas, color='coral')
plt.title("Grafik 7 Angka Hu Moments")

plt.show()
```

## HISTOGRAM OF ORIENTED GRADIENTS (HOG)

Histogram of Oriented Gradients (HOG) adalah sebuah metode ekstraksi fitur ( feature descriptor) yang sangat tangguh dalam visi komputer dan pengolahan citra, yang utamanya digunakan untuk deteksi objek. Metode ini pertama kali dipopulerkan oleh Navneet Dalal dan Bill Triggs pada tahun 2005 melalui penelitian mereka tentang deteksi pejalan kaki (pedestrian detection). Konsep dasar dari HOG adalah bahwa penampilan dan bentuk (shape) dari suatu objek lokal di dalam sebuah gambar dapat dideskripsikan dengan sangat baik oleh distribusi dari arah gradien intensitas (arah tepi/garis) pada area tersebut. Berbeda dengan LBP yang mencari pola ketetanggaan piksel, HOG berfokus pada perubahan drastis intensitas cahaya (gradien) untuk menangkap struktur kontur dan siluet objek, sekaligus mengabaikan informasi warna yang tidak relevan.

Langkah matematis pertama dalam algoritma HOG adalah menghitung nilai gradien dari citra. Citra berwarna umumnya diubah terlebih dahulu menjadi citra keabuan ( grayscale). Kemudian, citra tersebut disaring ( difilter) menggunakan kernel turunan orde pertama secara horizontal dan vertikal, biasanya menggunakan matriks $[-1, 0, 1]$ dan transpose-nya $[-1, 0, 1]^T$. Jika kita memiliki citra $I(x,y)$, maka gradien pada arah horizontal $G_x$ dan gradien pada arah vertikal $G_y$ untuk setiap piksel dihitung menggunakan persamaan:

$$G_x(x, y) = I(x + 1, y) - I(x - 1, y)$$

$$G_y(x, y) = I(x, y + 1) - I(x, y - 1)$$

Setelah nilai $G_x$ dan $G_y$ didapatkan, kita dapat menghitung kekuatan atau Magnitudo gradien ($m$) serta Orientasi/Sudut arah gradien ($\theta$) untuk setiap piksel menggunakan rumus trigonometri dasar:

$$m(x, y) = \sqrt{G_x(x, y)^2 + G_y(x, y)^2}$$

$$\theta(x, y) = \arctan\left(\frac{G_y(x, y)}{G_x(x, y)}\right)$$

Dalam implementasi standar HOG, sudut orientasi $\theta$ didefinisikan secara unsigned (tanpa tanda), yang berarti sudut berlawanan arah dianggap sama. Oleh karena itu, rentang sudut dibatasi hanya dari $0^\circ$ hingga $180^\circ$, bukan $0^\circ$ hingga $360^\circ$.

Langkah kedua adalah pembentukan histogram melalui proses yang disebut Spatial / Orientation Binning. Citra dibagi menjadi area-area kecil yang saling bersebelahan yang disebut Cells (Sel). Ukuran sel yang paling umum digunakan adalah $8 \times 8$ piksel. Untuk setiap sel, dibuat sebuah histogram orientasi gradien. Histogram ini umumnya terdiri dari 9 bins (keranjang) yang mewakili rentang sudut dari $0^\circ$ hingga $180^\circ$ (sehingga setiap bin memiliki lebar $20^\circ$, yaitu $0^\circ, 20^\circ, 40^\circ, \dots, 160^\circ$). Setiap piksel di dalam sel tersebut akan "memberikan suara" (vote) ke dalam bin histogram yang sesuai dengan sudut orientasi $\theta(x,y)$ miliknya. Nilai atau bobot suara yang dimasukkan ke dalam bin bukanlah angka 1, melainkan nilai magnitudo $m(x,y)$ dari piksel tersebut. Secara matematis, nilai histogram $h$ pada bin ke-$k$ di dalam sebuah sel dirumuskan sebagai:

$$h(k) = \sum_{(x,y)\in \text{cell}} m(x,y) \cdot \delta\big(\text{bin}\big(\theta(x,y)\big), k\big)$$

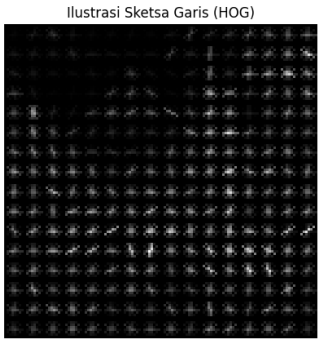
Di mana $\delta$ adalah fungsi indikator yang bernilai 1 jika sudut $\theta$ masuk ke dalam kategori bin $k$, dan bernilai 0 jika tidak. Penggunaan magnitudo sebagai pembobot memastikan bahwa tepi/garis yang sangat tajam dan jelas akan memberikan kontribusi (nilai) yang jauh lebih besar pada histogram dibandingkan area yang datar atau noise.

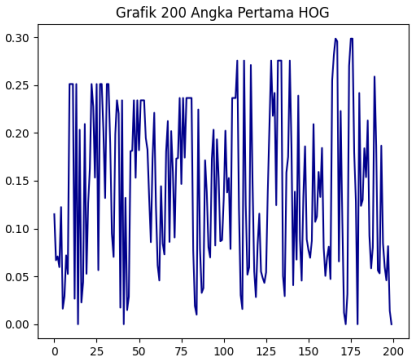
Langkah ketiga yang sangat krusial dalam HOG adalah Block Normalization (Normalisasi Blok). Nilai gradien sangat rentan terhadap perubahan pencahayaan lokal (iluminasi) dan kontras gambar. Untuk membuat fitur ini kebal terhadap perubahan cahaya (illumination invariant), histogram pada sel-sel tersebut harus dinormalisasi. Alih-alih menormalisasi setiap sel secara individual, HOG mengelompokkan beberapa sel yang berdekatan menjadi area yang lebih besar yang disebut Blocks (Blok). Ukuran blok yang umum adalah $2 \times 2$ sel (berisi 4 sel). Karena setiap sel memiliki 9 bins, satu blok akan memiliki vektor fitur $v$ dengan panjang $4 \times 9 = 36$ dimensi. Normalisasi dilakukan menggunakan perhitungan L2-norm (panjang Euclidean) dari vektor blok tersebut, dengan persamaan:

$$v_{norm} = \frac{v}{\sqrt{|v|_2^2 + \epsilon^2}}$$

Dalam persamaan di atas, $v$ adalah vektor tak ternormalisasi dari blok tersebut, $\|v\|_2$ adalah L2-norm yang dihitung dengan menjumlahkan kuadrat dari setiap elemen vektor $\left( \sqrt{\sum v_i^2} \right)$, dan $\epsilon$ adalah sebuah konstanta numerik yang sangat kecil (misalnya $10^{-5}$) yang ditambahkan untuk mencegah terjadinya galat pembagian dengan nol ( division by zero) apabila suatu area benar-benar kosong atau hitam pekat.

Hal penting yang perlu dicatat dari proses pembentukan blok ini adalah bahwa blok-blok tersebut dibuat saling tumpang tindih (overlapping). Biasanya, sebuah blok bergeser sejauh satu sel dalam satu waktu. Akibat tumpang tindih ini, setiap sel akan berkontribusi pada beberapa blok yang berbeda, dan dengan demikian nilai histogram dari satu sel yang sama akan dinormalisasi beberapa kali dengan faktor pencahayaan dari blok tetangga yang berbeda-beda. Langkah terakhir dalam HOG adalah menggabungkan ( concatenating) seluruh vektor blok yang telah dinormalisasi ($v_{norm}$) dari seluruh gambar menjadi satu larik (array) vektor satu dimensi yang sangat panjang. Vektor inilah yang menjadi HOG Feature Descriptor final. Vektor fitur yang sangat merepresentasikan bentuk fisik objek ini kemudian diumpankan ke dalam algoritma Machine Learning, di mana algoritma Linear Support Vector Machine (SVM) menjadi pasangan paling klasik dan tangguh untuk melakukan klasifikasi gambar berdasarkan fitur HOG tersebut.




**CODE**

```
# 1. MENYAMAKAN UKURAN GAMBAR (RESIZE)
# Agar komputernya tidak bingung, semua gambar harus diubah ke ukuran baku
(seperti pas foto)
ukuran_baru = (128, 128)
gambar_pas_foto = cv2.resize(gambar_abu, ukuran_baru)

# 2. PENGATURAN SKETSA (HOG)
jumlah_arah = 9          # Kita bagi arah garis menjadi 9 sudut (tegak,
datar, miring, dst)
ukuran_sel = (8, 8)      # Gambar dibagi jadi kotak-kotak kecil ukuran 8x8
piksel
ukuran_blok = (2, 2)     # Kotak kecil tadi digabung jadi blok 2x2 agar
kebal terhadap bayangan/cahaya gelap

# 3. MENGHITUNG DAN MENGGAMBAR HOG
# Mesin akan mengeluarkan 2 hal:
# a. Barisan angka fitur (untuk disetor ke AI)
# b. Gambar ilustrasi sketsanya (karena kita set visualize=True)
angka_hog, gambar_garis_hog = hog(
    gambar_pas_foto,
```

```
    orientations=jumlah_arah,
    pixels_per_cell=ukuran_sel,
    cells_per_block=ukuran_blok,
    visualize=True,       # Minta digambarkan ilustrasinya
    feature_vector=True   # Minta angkanya diratakan jadi satu baris panjang
)

# 4. MENAMPILKAN HASIL AKHIR (TEKS)
print("Laporan Garis Tepi (HOG):")
print("--------------------------------------------------")
print(f" - Ukuran Gambar Baku : {ukuran_baru} piksel")
print(f" - Total Angka Didapat: {len(angka_hog)} angka")
print(f" - Nilai Terkecil     : {angka_hog.min():.4f}")
print(f" - Nilai Terbesar     : {angka_hog.max():.4f}")
print("--------------------------------------------------")

# 5. MENGGAMBAR KE LAYAR
plt.figure(figsize=(20, 5)) # Siapkan 3 kanvas

# --- Tampilan 1: Gambar Pas Foto (Resized) ---
plt.subplot(1, 3, 1)
plt.imshow(gambar_pas_foto, cmap='gray')
plt.title(f"Gambar Ukuran Baku {ukuran_baru}")
plt.axis("off")

# --- Tampilan 2: Gambar Sketsa HOG ---
plt.subplot(1, 3, 2)
plt.imshow(gambar_garis_hog, cmap='gray')
plt.title("Ilustrasi Sketsa Garis (HOG)")
plt.axis("off")

# --- Tampilan 3: Grafik Barisan Angka ---
plt.subplot(1, 3, 3)
# Angka HOG itu jumlahnya ribuan. Kalau digambar semua grafiknya jadi ruwet.
# Jadi kita ambil 200 angka pertama saja (pakai [:200]) untuk sekadar
dilihat bentuknya.
plt.plot(angka_hog[:200], color='darkblue')
plt.title("Grafik 200 Angka Pertama HOG")

plt.show()
```

## EDGE FEATURES (CANNY + SOBEL)

Edge Features (Fitur Tepi) adalah salah satu jenis fitur tingkat rendah (*low-level features*) yang paling fundamental dalam pengolahan citra digital dan visi komputer. Tepi ( *edge*) didefinisikan sebagai area dalam citra di mana terjadi perubahan intensitas cahaya yang sangat drastis atau diskontinuitas secara tiba-tiba. Ekstraksi garis tepi sangat penting karena tepi mewakili batas struktural dari suatu objek, memisahkan objek dari latar belakangnya, serta menangkap bentuk dasar ( *shape*) geometrisnya sambil membuang informasi yang seringkali tidak relevan seperti warna pekat atau tekstur permukaan yang seragam. Dua metode paling populer untuk mengekstrak fitur tepi ini adalah Operator Sobel dan Algoritma Canny.

Operator Sobel adalah metode deteksi tepi berbasis turunan orde pertama ( *first-order derivative*). Konsep matematikanya berakar pada perhitungan gradien intensitas citra secara spasial. Karena citra digital adalah fungsi diskrit $I(x,y)$, turunan tidak dihitung secara kontinu, melainkan diaproksimasi menggunakan operasi konvolusi dengan matriks kernel berukuran $3 \times 3$. Sobel menggunakan dua kernel khusus: satu untuk mendeteksi perubahan intensitas pada arah horizontal ($G_x$) dan satu lagi untuk arah vertikal ($G_y$). Kernel Sobel memberikan bobot lebih besar pada piksel tengah untuk memberikan efek penghalusan (*smoothing*) bawaan yang membantu mengurangi derau (*noise*). Secara matematis, matriks kernel Sobel didefinisikan sebagai:

$$K_x = \left[\!\left[ \{bmatrix\} -1 \& 0 \& 1 \backslash\backslash -2 \& 0 \& 2 \backslash\backslash -1 \& 0 \& 1 \right]\!\right] \{bmatrix\}, \quad K_y = \left[\!\left[ \{bmatrix\} -1 \& -2 \& -1 \backslash\backslash 0 \& 0 \& 0 \backslash\backslash 1 \& 2 \& 1 \right]\!\right] \{bmatrix\}$$

Untuk mendapatkan nilai gradien pada citra $I$, kita melakukan konvolusi ($*$) antara citra dengan masing-masing kernel tersebut: $G_x = K_x * I$ dan $G_y = K_y * I$.

Setelah nilai gradien horizontal dan vertikal didapatkan untuk setiap piksel, kita dapat menghitung Magnitudo Gradien ($|G|$) dan Arah Gradien ($\theta$). Magnitudo menunjukkan seberapa kuat tepi tersebut (semakin besar nilainya, semakin tajam tepinya), sedangkan arah gradien selalu tegak lurus terhadap arah garis tepi itu sendiri. Persamaan matematis untuk magnitudo dan arah adalah:

$$|G| = \sqrt{G_x^2 + G_y^2}$$

$$\theta = \arctan\left(\frac{G_y}{G_x}\right)$$

Dalam praktiknya, untuk mempercepat komputasi, magnitudo sering diaproksimasi dengan nilai absolut jumlahan gradien: $|G| \approx |G_x| + |G_y|$. Operator Sobel sangat cepat dan efektif, namun memiliki kelemahan utama: garis tepi yang dihasilkan sering kali tebal, kabur, dan sangat sensitif terhadap *noise*, sehingga kurang presisi jika digunakan langsung sebagai fitur bentuk.

Untuk mengatasi kelemahan Sobel, John F. Canny pada tahun 1986 memperkenalkan Canny Edge Detector. Canny bukanlah sekadar operator matematika sederhana, melainkan sebuah algoritma *multi-stage* (multi-tahap) yang dirancang untuk menjadi detektor tepi optimal. Tahap pertama dari Canny adalah Noise Reduction (Pengurangan Derau). Karena deteksi tepi sangat sensitif terhadap *noise*, citra terlebih dahulu dihaluskan menggunakan *Gaussian Filter*. Operasi ini melibatkan konvolusi citra dengan kernel Gaussian 2D, yang persamaan matematisnya adalah:

$$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Di mana $\sigma$ adalah standar deviasi yang menentukan tingkat kehalusan. Setelah citra dihaluskan, tahap kedua adalah menghitung gradien magnitudo dan arahnya menggunakan Operator Sobel (seperti yang telah dijelaskan pada paragraf sebelumnya).

Tahap ketiga, yang menjadi kunci ketajaman Canny, adalah Non-Maximum Suppression (NMS). NMS adalah teknik *edge thinning* (penipisan tepi). Algoritma ini memindai seluruh citra dan memeriksa setiap piksel untuk melihat apakah piksel tersebut merupakan nilai maksimum lokal di lingkungan sekitarnya *searah dengan arah gradiennya* ($\theta$). Misalnya, jika arah gradien suatu piksel mendekati $0^\circ$ (tepi vertikal), maka magnitudo piksel tersebut dibandingkan dengan piksel di sebelah kiri dan kanannya. Jika magnitudonya lebih besar dari kedua tetangganya, piksel tersebut dipertahankan (nilainya tetap); jika tidak, nilainya ditekan menjadi 0 (dihapus). Proses ini memastikan bahwa garis tepi yang dihasilkan hanya memiliki ketebalan tepat satu piksel ( *single-pixel width*).

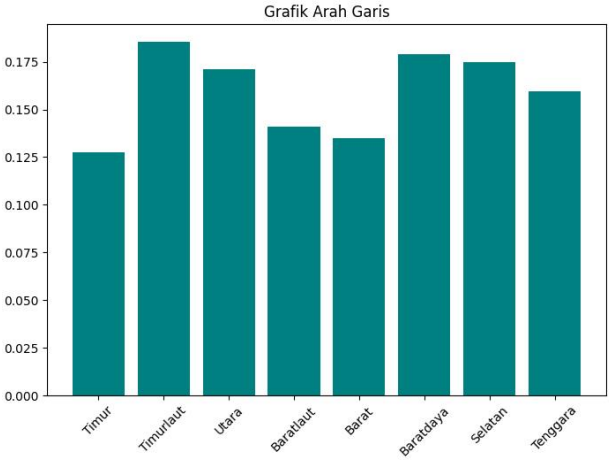
Tahap terakhir dari Canny adalah Hysteresis Thresholding (Ambang Batas Histeresis). Canny menggunakan dua nilai ambang batas: *High Threshold* ($T_{high}$) dan *Low Threshold* ($T_{low}$). Tujuannya adalah untuk memisahkan tepi yang benar-benar nyata dari tepi palsu yang mungkin diakibatkan oleh variasi warna kecil. Aturannya adalah sebagai berikut:

Jika $|G| \geq T_{high}$, piksel diklasifikasikan sebagai tepi kuat (*strong edge*).

Jika $|G| < T_{low}$, piksel ditekan/dibuang menjadi 0.

Jika $T_{low} \leq |G| < T_{high}$, piksel diklasifikasikan sebagai tepi lemah (*weak edge*).

Sebuah tepi lemah hanya akan dipertahankan sebagai tepi yang sah jika dan hanya jika ia terhubung (bertetangga secara 8-konektivitas) dengan piksel tepi kuat. Pendekatan ini mencegah terputusnya garis tepi ( *fragmentation*) akibat fluktuasi *noise* kecil. Citra biner yang dihasilkan dari algoritma Canny ini memberikan peta kontur objek yang sangat bersih, tipis, dan presisi, yang selanjutnya dapat digunakan secara langsung sebagai fitur masukan untuk ekstraksi kontur, transformasi Hough (untuk deteksi garis/lingkaran), atau pencocokan *template* (*template matching*) dalam sistem visi komputer.



## CODE

```
# 1. MEMBUAT SKETSA GARIS TEPI (CANNY)
batas_bawah = 50   # Abaikan garis yang terlalu tipis/samar
batas_atas = 150   # Pastikan ambil garis yang tebal/tegas

# Mesin menggambar ulang fotomu menjadi sketsa garis hitam putih
garis_tepi = cv2.Canny(gambar_abu, batas_bawah, batas_atas)

# 2. MENDETEKSI PERUBAHAN WARNA (SOBEL / GRADIEN)
# CV_64F itu ibarat kita menyuruh mesin: "Tolong hitung pakai angka desimal
ya, biar akurat"
# Mesin memindai/scan gambar dari Kiri ke Kanan (Mendatar)
scan_mendatar = cv2.Sobel(gambar_abu, cv2.CV_64F, 1, 0, ksize=3)

# Mesin memindai gambar dari Atas ke Bawah (Menurun)
scan_menurun = cv2.Sobel(gambar_abu, cv2.CV_64F, 0, 1, ksize=3)

# 3. MENGHITUNG KEKUATAN & ARAH GARIS (RUMUS MATEMATIKA)
# Menggabungkan hasil scan pakai rumus Pythagoras
kekuatan_garis = np.sqrt(scan_mendatar2 + scan_menurun2)

# Mencari arah derajat miringnya garis (seperti kompas)
arah_garis = np.arctan2(scan_menurun, scan_mendatar)

# 4. MERINGKAS HASIL MENJADI 5 ANGKA STATISTIK
# a. Kepadatan: Berapa persen isi fotomu yang berupa garis tepi?
kepadatan_garis = np.sum(garis_tepi > 0) / garis_tepi.size

# b & c. Rata-rata kekuatan garis dan variasinya
rata2_kekuatan = np.mean(kekuatan_garis)
variasi_kekuatan = np.std(kekuatan_garis)

# d & e. Rata-rata arah garis dan variasinya
rata2_arah = np.mean(arah_garis)
variasi_arah = np.std(arah_garis)

# Kita kumpulkan ke keranjang
keranjang_angka = [kepadatan_garis, rata2_kekuatan, variasi_kekuatan,
rata2_arah, variasi_arah]

# 5. MEMBUAT GRAFIK KOMPAS ARAH GARIS (8 ARAH)
# Kita kelompokkan garis-garis tadi ke dalam 8 arah mata angin (Timur,
Barat, Utara, dst)
grafik_kompas, _ = np.histogram(arah_garis.flatten(), bins=8, range=(-np.pi,
np.pi), density=True)

# Extend ke keranjang kita
keranjang_angka.extend(grafik_kompas)

# 6. MENAMPILKAN HASIL TEKS & GAMBAR
hasil_akhir = np.array(keranjang_angka)

print("Laporan Deteksi Garis & Tepi:")
print("--------------------------------------------------")
print(f" - Kepadatan Garis      : {kepadatan_garis*100:.1f} % dari total
gambar")
```

```
print(f" - Rata-rata Kekuatan   : {rata2_kekuatan:.4f}")
print(f" - Rata-rata Arah       : {rata2_arah:.4f} radian")
print(f" - Jumlah Kelompok Arah : 8 arah mata angin")
print(f" - Total Angka Dihasilkan: {len(hasil_akhir)} angka")
print("   (5 statistik utama + 8 angka arah mata angin)")
print("--------------------------------------------------")

# Menyiapkan 4 Kanvas
plt.figure(figsize=(18, 12))

# --- Tampilan 1: Gambar Asli ---
plt.subplot(2, 2, 1)
plt.imshow(gambar_abu, cmap='gray')
plt.title("Gambar Asli")
plt.axis("off")

# --- Tampilan 2: Sketsa Canny ---
plt.subplot(2, 2, 2)
plt.imshow(garis_tepi, cmap='gray')
plt.title("Sketsa Tepi (Canny)")
plt.axis("off")

# --- Tampilan 3: Peta Kekuatan Garis ---
plt.subplot(2, 2, 3)
plt.imshow(kekuatan_garis, cmap='hot')
plt.title("Peta Kekuatan Garis (Gradient Magnitude)")
plt.axis("off")

# --- Tampilan 4: Grafik Arah Garis ---
plt.subplot(2, 2, 4)
nama_arah = ['Timur', 'Timurlaut', 'Utara', 'Baratlaut', 'Barat',
'Baratdaya', 'Selatan', 'Tenggara']
plt.bar(nama_arah, grafik_kompas, color='teal')
plt.title("Grafik Arah Garis")
plt.xticks(rotation=45) # Tulisan dimiringkan sedikit agar tidak tabrakan

plt.show()
```